# UCS CRM - Database Security Review

# Database Security Review

**Scope:** The AWS RDS PostgreSQL data store that backs UCS CRM, its credentials, connection settings, exposed query endpoints, and supporting infrastructure (backups, Redis, S3).

**Date:** 2026-10-09 **Method:** Read the source and run `backend/scripts/security-check.mjs`. Every finding below cites the file and line that proves it.

**Rule:** where `file:line` is given, the code is the source of truth, not this document. Rotate first, then fix, then verify.

---

## 1. Summary

| ID | Severity | Finding | Status |
|---|---|---|---|
| DB-001 | **Critical** | Production database passwords committed in tracked files | Open |
| DB-002 | **Critical** | `/api/db/*` query endpoints are unauthenticated | Open |
| DB-003 | **High** | Application connects as the master ( `ucs_admin` ) DB user | Open |
| DB-004 | **High** | Database TLS certificate verification is disabled | Open |
| DB-005 | **High** | RDS is publicly reachable; security group is auto-opened | Open |
| DB-006 | **Medium** | No row-level security; no defence in depth | Open |
| DB-007 | **Medium** | Weak JWT fallback, 100-year tokens, default passwords | Open |
| DB-008 | **Medium** | Aadhaar/PAN stored in plaintext; no masking or retention | Open |
| DB-009 | **Medium** | Backups not proven encrypted at rest or restorable | Open |
| DB-010 | Low | `/db-viewer` and `/env-admin` HTML served unauthenticated | Open |

Fix order: **DB-001** → **DB-002** → **DB-003** → **DB-004** → **DB-005**, then the rest.

---

## 2. Findings

### DB-001 — Production database passwords committed in tracked files

**Severity:** Critical · Confirmed · Tracked in git

`npm run security:check` fails with nine Postgres DSNs that carry a real password, plus a WhatsApp token, an AWS key and a `VITE_` secret. The database findings:

```
SN-01  Postgres connection string with an inline password  (9)
    backend/scripts/cleanup_star.mjs:2                xm4**************qLe
    backend/scripts/debug-leads-names.cjs:2           Sev****32P
    backend/scripts/debug-leads-names.mjs:2           Sev****32P
    backend/scripts/delete-receipt-82104.cjs:5        Sev****32P
    backend/scripts/migration-station-fro/01-dedupe.mjs:16        ucs*******123
    backend/scripts/migration-station-fro/02-excel-stations.mjs:22 ucs*******123
    backend/scripts/migration-station-fro/03-write-through.mjs:19  ucs*******123
    backend/scripts/migration-station-fro/04-verify.mjs:10         ucs*******123
    backend/scripts/sync-down/sync-down.mjs:16        Sev****32P
```

Two accounts are exposed: `ucs_admin` (master/owner) and `ucs_app`. `git ls-files` confirms every one of these files is tracked.

**Impact:** Anyone with the repository, or a leaked copy of it, holds full control of the production database. Deleting the lines is **not** enough — git history keeps them forever, so the credentials must be treated as public.

**Remediation** 1. **Rotate now** (before touching the code): `sql  ALTER ROLE ucs_app WITH PASSWORD '<new-strong-random>';  -- master: AWS Console -> RDS -> Modify -> New master password` 2. Stop hardcoding. Read the value from the environment: `js  const c = new Client({ connectionString: process.env.DATABASE_URL });` 3. Scrub git history (after rotation): `bash  pip install git-filter-repo  # replacements.txt:  #  Sevak1432P==>***REMOVED***  #  ucscompany123==>***REMOVED***  #  xm4BAoZRhOVU3NFW2qLe==>***REMOVED***  git filter-repo --replace-text replacements.txt` 4. Gate it so it cannot recur: run `npm run security:check` in CI and as a pre-commit hook. It must be red until this is done.

---

## DB-002 — `/api/db/*` query endpoints are unauthenticated

**Severity:** Critical · Confirmed

These routes run against production with no auth middleware of any kind ( `backend/src/index.js` ):

| Route | Line | What it does |
|---|---|---|
| `GET /api/db/tables` | 557 | List every table + row estimate |
| `GET /api/db/table/:table` | 573 | Read any table (schema + rows) |
| `POST /api/db/query` | 696 | **Run arbitrary SQL** (multi-statement) |
| `POST /api/db/drop-table` | 733 | Drop any table |
| `POST /api/db/rows/delete` | 746 | Delete rows by PK |
| `POST /api/db/rows/update` | 795 | Update rows by PK |
| `GET /db-viewer` (page) | 549 | The HTML tool |

Only `GET /api/db/media` (line 861) is guarded, with `requireAdminKey`. `POST /api/db/query` wraps statements in a transaction and can run `DROP`, `UPDATE`, `DELETE`, `COPY`, etc.

**Impact:** Anyone on the internet can read, modify or destroy the entire database without a token. This is the single most dangerous defect in the database layer.

**Remediation:** gate every route (behind both a role and the admin key, and ideally only outside production):

```
import { authenticateRole } from './middleware/authMiddleware.js';
const opsOnly = [requireAdminKey, authenticateRole('super_admin', 'developer')];

app.get('/api/db/tables',        ...opsOnly, async (req, res) => { /* ... */ });
app.get('/api/db/table/:table',  ...opsOnly, async (req, res) => { /* ... */ });
app.post('/api/db/query',        ...opsOnly, async (req, res) => { /* ... */ });
app.post('/api/db/drop-table',   ...opsOnly, async (req, res) => { /* ... */ });
app.post('/api/db/rows/delete',  ...opsOnly, async (req, res) => { /* ... */ });
app.post('/api/db/rows/update',  ...opsOnly, async (req, res) => { /* ... */ });
app.get('/db-viewer',            ...opsOnly, (req, res) => { /* ... */ });
```

Note: `requireAdminKey` currently fails **open** when `ENV_ADMIN_KEY` is unset (see DB-010), so fix that too.

---

### DB-003 — Application connects as the master (`ucs_admin`) DB user

**Severity:** High · Confirmed

The app reaches the database through `DATABASE_URL` (`backend/src/config/db.js:28-41`). Operational evidence shows the master account is in use (`ucs_admin` appears in the tracked scripts and restore tooling, e.g. `backend/scripts/backup/restore-test.mjs:112`). A separate lower-privilege role `ucs_app` exists but the application is not restricted to it.

**Impact:** A master/owner account can read other schemas, manage roles and run `DROP DATABASE`. Any SQL injection or compromised process becomes total database compromise.

**Remediation:** run the application as a least-privilege role; reserve the master for migrations.

```
GRANT USAGE ON SCHEMA public TO ucs_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO ucs_app;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO ucs_app;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO ucs_app;
REVOKE CREATE ON SCHEMA public FROM ucs_app;
```

Then point `DATABASE_URL` at `ucs_app`. Run migrations with the admin role.

---

### DB-004 — Database TLS certificate verification is disabled

**Severity:** High · Confirmed

Every Postgres connection sets `ssl: { rejectUnauthorized: false }`:

- `backend/src/config/db.js:31` and `:39`
- `backend/src/models/workerModel.js:188`
- `backend/src/routes/tempCleanupRoutes.js:13` and `:21`
- `backend/src/services/customerProvision.js:78`

**Impact:** The client accepts any certificate, so an on-path attacker can intercept or alter database traffic. This matters more than usual because the RDS instance is publicly reachable (DB-005).

**Remediation:** pin the RDS CA bundle and verify:

```
import fs from 'fs';
poolConfig.ssl = {
  ca: fs.readFileSync(process.env.RDS_CA_PATH || './global-bundle.pem'),
  rejectUnauthorized: true,
};
```

Bundle: `https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem`. Apply the same change in the four other files above.

---

### DB-005 — RDS is publicly reachable; security group is auto-opened

**Severity:** High · Confirmed

`backend/src/middleware/localDbAccess.js` authorizes the machine's current public IP on port 5432 at boot and revokes it on shutdown. The guard logic is careful (single-host `/32`, TTL sweep, tagged rules), but the design assumes a publicly accessible RDS instance (`instance.PubliclyAccessible`, checked at `localDbAccess.js:382`).

**Impact:** The database port is exposed to the internet. A leftover rule after a crash, a wide CIDR a human adds, or a leaked `ucs_admin` credential (DB-001) is directly reachable.

**Remediation** - Move RDS into private subnets and set `PubliclyAccessible = false`. - Reach it via the SSH tunnel (`backend/start-local-db-tunnel.ps1`) or a VPN. - Keep `LOCAL_DB_ACCESS=false` outside local development. - Restrict the security group to the application security group only. - Rotate the AWS access key that is itself committed (see security-check `SN-03` in `others/imp files/docs/BACKUP_DISASTER_RECOVERY.md:381`).

---

### DB-006 — No row-level security; no defence in depth

**Severity:** Medium · Confirmed

No migration enables RLS or creates a policy (search for `CREATE POLICY` / `ENABLE ROW LEVEL SECURITY` returns nothing across `backend/**/*.sql`). Authorisation lives entirely in application code — and DB-002 shows some routes have none.

**Impact:** One missing role check is a full data breach rather than a single endpoint's data exposure. Tenant isolation (NGOs) relies on every query being written correctly.

**Remediation:** after DB-003 (least-privilege role), enable RLS on the sensitive tables (`beneficiaries`, `workers`, receipt/donor tables) with policies keyed on `ngo_id`/owner, so the database itself refuses cross-tenant reads.

---

### DB-007 — Weak JWT fallback, 100-year tokens, default passwords

**Severity:** Medium · Confirmed (affects who can reach the database)

- `backend/src/config/db.js:1300` — `process.env.JWT_SECRET || 'dev-secret'` signs tokens with a publicly known secret if the env var is missing.
- `backend/src/config/db.js:1301` — the same token is issued with `expiresIn: '100y'`.
- `backend/src/models/crmAgentModel.js:20` and `backend/src/controllers/workerController.js:46` — default password `'123456'`.
- `backend/src/controllers/authController.js:222-223` — admin login compares `ADMIN_PASSWORD` with plain `===`.

**Impact:** A forged or long-lived token reaches every authenticated DB route, including the ones that should be admin-only.

**Remediation:** remove the `'dev-secret'` fallback (fail fast at boot if `JWT_SECRET` is unset), bound every token lifetime, force a password change on first login, and store admin credentials as a hash.

---

### DB-008 — Aadhaar/PAN stored in plaintext; no masking or retention

**Severity:** Medium · Confirmed

Aadhaar and PAN live in plaintext columns (`workers`, `beneficiaries`, donor records) with no application-level encryption, no masking in the UI, and no retention/deletion job in code.

**Impact:** A single database read exposes the most sensitive identifiers the organisation holds. This is the highest-value data in the system.

**Remediation:** encrypt these columns (pgcrypto or application-level), mask them in every read path with a reveal gate, and implement a retention delete as part of the DPDP Act 2023 obligations.

---

### DB-009 — Backups not proven encrypted at rest or restorable

**Severity:** Medium · Partially confirmed

Infrastructure exists: AWS Backup vault `ucs-crm-backup-vault`, plan `ucs-crm-rds-backup-plan`, S3 bucket `ucs-crm-backups`, and `backend/scripts/backup/restore-test.mjs`. The README lists the intended security checklist but there is no recorded proof it is applied or that a restore was tested.

**Impact:** A backup containing Aadhaar and salary is as sensitive as the live database. An untested backup is not a backup.

**Remediation:** verify KMS encryption on the vault and S3 bucket, run `restore-test.mjs` on a schedule, keep the RDS password in Secrets Manager, and record each restore test.

---

### DB-010 — `/db-viewer` and `/env-admin` HTML served unauthenticated

**Severity:** Low · Confirmed

- `GET /db-viewer` (`backend/src/index.js:549`) and `GET /env-admin` (`:553`) send the HTML pages with no guard.
- `backend/src/middleware/requireAdminKey.js:7-11` and `backend/src/routes/envAdminRoutes.js:37-44` both **fail open** when `ENV_ADMIN_KEY` is unset — the check passes and the endpoints are open.

**Impact:** The env-admin surface can read and write every project's `.env` and restart services. Failing open means a missing env var silently disables the guard.

**Remediation:** make the guard fail closed (`if (!ADMIN_KEY) return res.status(503)...`), guard the HTML pages, and require the key for env-admin in production.

---

## 3. Immediate action plan

| # | Action | Finding | Why first |
|---|---|---|---|
| 1 | Rotate `ucs_admin`, `ucs_app`, the AWS key and the WhatsApp token | DB-001 | The values are already public |
| 2 | Add auth to every `/api/db/*` route and `/db-viewer` | DB-002 | Live arbitrary-SQL exposure |
| 3 | Point `DATABASE_URL` at `ucs_app`; grant least privilege | DB-003 | Limits blast radius |
| 4 | Pin the RDS CA and set `rejectUnauthorized: true` | DB-004 | Stops MITM |
| 5 | Private RDS + SSH tunnel; keep `LOCAL_DB_ACCESS=false` | DB-005 | Removes internet exposure |
| 6 | Scrub git history; enforce `security:check` in CI | DB-001 | Prevents recurrence |
| 7 | Make `requireAdminKey` fail closed | DB-010 | Closes a silent bypass |
| 8 | Bound token lifetimes; remove `dev-secret`; drop default `123456` | DB-007 | Reduces valid credentials |
| 9 | Enable RLS on sensitive tables | DB-006 | Defence in depth |
| 10 | Encrypt/mask Aadhaar & PAN; retention job | DB-008 | Highest-value data |
| 11 | Verify backup encryption; schedule restore test | DB-009 | Recoverability |

## 4. Verification

```
# Credentials must not appear in tracked files. Red = unfixed.
cd backend && npm run security:check

# Confirm which files are tracked.
git ls-files | grep -E 'scripts/(sync-down|cleanup_star|debug-
        leads|fro_parity|receipt_gap)'
```

```
# Confirm the DB routes carry a guard after the fix.
grep -n "api/db/" backend/src/index.js
```